Big Data Pipeline for Automated Video Transcription and TF-IDF Keyword Extraction

Manav Jain, Heetej Jagatia
DTSC303 Big Data Computing

April 13, 2026

Abstract

In the era of exponential digital media growth, unstructured video data represents a significant analytical blind spot for content platforms. This project outlines the engineering and deployment of a distributed Big Data pipeline designed to automate the ingestion, transcription, and contextual tagging of raw video files. Utilizing Apache Spark on an Amazon Elastic MapReduce (EMR) cluster, the system parallelizes the workload across multiple nodes. It leverages OpenAI's Whisper artificial intelligence model for audio transcription and applies the Term Frequency-Inverse Document Frequency (TF-IDF) mathematical model to extract highly relevant metadata. The resulting architecture successfully demonstrates horizontal scalability, fault-tolerant dependency management, and the automated generation of Search Engine Optimization (SEO) tags from unstructured media.

Contents

1 Introduction and Problem Statement

Related Works, Refernces, Any data put in github.

Video files (such as .mp4, .mov, or .avi) are notoriously difficult to index. Search engines and databases inherently rely on text-based queries. When an educational platform hosts tens of thousands of video lectures, locating the exact timestamp where a professor discusses a specific concept is practically impossible without manual human transcription and tagging. To resolve this bottleneck, this project produces a solution by engineering a fully automated pipeline that converts video into raw audio into accurate text transcripts and mathematically deduces core subjects using natural language processing (TF-IDF in this case), removing the need for manual human intervention.

2 Technology Stack and Infrastructure

To handle the computationally heavy task of transcription and natural language processing at scale, the pipeline was built using a cloud-native technology stack:

- **Amazon Web Services (AWS) S3:** Served as the Data Lake and bucket for staging raw files and storing our outputs.
- **AWS Elastic MapReduce (EMR):** Provided the cloud cluster infrastructure for our nodes.
- **Apache Spark (PySpark):** Was the preferred distributed framework. Spark's in-memory processing drastically reduced the time required for NLP tasks compared to traditional MapReduce.
- **OpenAI Whisper:** An open-source Automatic Speech Recognition (ASR) model used to transcribe audio to text with high accuracy.

The whole pipeline was constructed in python with common operational libraries along with pyspark for Apache Spark access, boto3 for S3 access and numpy for basic mathematical calculations. The output is stored in the s3 in parquet, ideal for the apache framework, and in csv for user output and convenience.

3 System Architecture and Pipeline Methodology

The data lifecycle within this pipeline moves through four phases.

3.1 Phase 1: Data Ingestion and Staging

Raw video files are uploaded into a designated Amazon S3 bucket. This decoupling of storage from compute ensures data persistence even if the EMR cluster is terminated. Spark reads the directory structure and generates an initial Resilient Distributed Dataset (RDD) containing the file paths.

3.2 Phase 2: Distributed AI Transcription

The Spark master node delegates individual video files to available worker nodes, where the Whisper model, relying on `ffmpeg`, extracts the audio tracks.

3.3 Phase 3: Natural Language Processing and Data Cleaning

Once the audio is converted to raw text, Spark's machine learning library (`pyspark.ml`) is utilized to process the data. The text undergoes tokenization and normalization.

3.4 Phase 4: Data Aggregation and Export

The final phase prepares the distributed data for users to then use in business intelligence tools for analysis of the data. For this, the final output is aggregated as a csv file.

3.5 Phase 5: Search and Retrieval

To the extracted metadata, a lightweight PySpark search mechanism was implemented. Leveraging PySpark's `array_contains` function, a querying script was built to instantly filter the distributed JSON arrays. Users can pass a keyword directly into the terminal command, and the engine will instantly scan the S3 data lake to return the exact video file where that topic is discussed.

The terminal provides a clean, database-style output of the matched files. This querying capability effectively turns the static data lake into an interactive, searchable content recommendation engine.

4 Mathematical Model: TF-IDF

To extract the most important topics rather than simply the most frequent words, the system utilizes the Term Frequency-Inverse Document Frequency (TF-IDF) algorithm.

First, the system calculates the **Term Frequency (TF)**:

$$TF = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of words in the document}} \tag{1}$$

Next, it calculates the **Inverse Document Frequency (IDF)**, which penalizes common words:

$$IDF = \log\left(\frac{N}{df_t}\right) \tag{2}$$

Where N is the total number of documents/videos, and df_t is the number of documents containing the term t.

The final contextual weight is:

$$TF\text{-}IDF = TF \times IDF \tag{3}$$

5 Results and Discussion

The fundamental advantage of this architecture is its horizontal scalability. Because Spark distributes tasks across nodes, the system is enterprise-ready and capable of handling thousands of videos without requiring a codebase rewrite.

The pipeline successfully processed a diverse sample dataset and extracted accurate contextual tags:

- **Biology Lecture:** carbon, bond, electrons, atoms, sodium, water
- **Historical Speech:** new, space, man, years, knowledge, moon
- **Informational Clip:** goats, fainting, muscles, excited, leg

6 Conclusion and Future Work

This project successfully engineered a cloud-native pipeline capable of unlocking the data within video files. The immediate business value lies in automated SEO, content recommendation, and improved accessibility. Future iterations could involve integrating Large Language Models (LLMs) to generate paragraph-length summaries alongside targeted keywords.

Following is the GitHub link to the final code repository:

https://github.com/ManavJainDev/VideoTranscriptionTFIDF.git